

# Text Region Detection Project

Project Report — Computer Vision Coursework  
European Humanities University, Vilnius | June 2026

| Team Member        | Role                            | Primary Modules                         |
|--------------------|---------------------------------|-----------------------------------------|
| Michail Odintsov   | Image Processing Specialist     | preprocessing/ (enhance.py, segment.py) |
| Maksim Darvin      | Morphology & Visualization Lead | morphology/ (clean.py), visualization/  |
| Aliaksei Banshchyk | Lead CV Engineer                | detection/ (detect.py, decide.py)       |

## 1. Problem Description

The goal of this project is to locate regions containing text within an arbitrary input image - for example, a speech bubble in a comic panel, a sign in a photograph, or an overlaid caption - using only classical (non deep-learning) computer vision techniques. The system does not need to recognise *what* the text says; it only needs to answer two questions reliably: **(1)** does this image contain text at all, and **(2)** if so, where are its bounding boxes located?

This is a deceptively hard problem for a purely rule-based pipeline because the same low-level cues that make text distinctive - sharp edges, local contrast, compact connected shapes - are also produced by non-text content: character outlines, fur or fabric texture, fence patterns, and busy backgrounds all generate similar gradients and blob shapes. A large part of the engineering effort documented in this report went into telling these two categories apart without any learned model.

— Section authored by Lead CV Engineer

## 2. Team Roles & Task Division

The pipeline was split into independent stages with a fixed input/output "contract" (a documented array shape and value range) between each pair of stages. This let each member work, test, and debug their module in isolation before integration.

| Member                                             | Owned Stages                                                  | Key Responsibilities                                                                                                                                                                                                             |
|----------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Michail Odintsov<br>(Image Processing Specialist)  | Stage 1 — Enhancement<br>Stage 2 — Segmentation               | Image loading, grayscale conversion, adaptive gamma correction, CLAHE, non-local means denoising; three segmentation strategies (adaptive threshold, Sobel gradient, Otsu) and their combination into one binary mask.           |
| Maksim Darvin<br>(Morphology & Visualisation Lead) | Stage 3 — Morphological Cleaning<br>Visualization & Reporting | Opening/erosion/dilation/closing chain to remove noise and merge characters into words; connected-component filtering; building the six-panel stage-grid visualisation and the contact sheets used for the demo and this report. |

|                                          |                                                    |                                                                                                                                                                                                                       |
|------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aliaksei Banshchyk<br>(Lead CV Engineer) | Stage 4 —<br>Detection Stage 5<br>— Decision Logic | Contour-to-bounding-box conversion, geometry filtering,<br>Non-Maximum Suppression, isolated-noise filtering; final<br>text/no-text verdict logic, region classification<br>(word/line/block) and JSON result export. |
|------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Integration was coordinated through documented "input/output contracts" at each module boundary (e.g. Stage 2 always hands Stage 3 a uint8 mask with values restricted to {0, 255}), so any member could unit-test their stage against synthetic masks without waiting on a teammate's code being finished.

— Section authored by Morphology & Visualization Lead

### 3. Pipeline Design

The system is a strictly linear five-stage pipeline. Each stage consumes the previous stage's output and is independently swappable — for example, Stage 2 supports three interchangeable segmentation strategies behind one function signature.

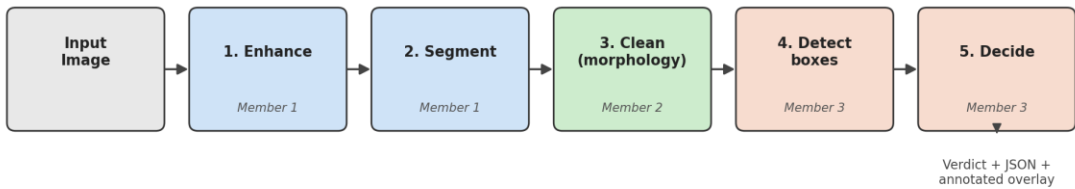


Figure 1. Five-stage pipeline architecture and module ownership.

**Stage 1 — Enhancement:** normalises lighting and contrast so the same thresholds work across dark photos and bright scans. **Stage 2 — Segmentation:** produces a binary candidate mask of "text-like" pixels. **Stage 3 — Morphological cleaning:** removes noise speckles and merges individual character strokes into solid word/line blobs. **Stage 4 — Detection:** converts surviving blobs into bounding boxes and discards boxes that fail geometric sanity checks. **Stage 5 — Decision:** aggregates the surviving boxes into a single text-found / no-text verdict and a structured JSON result.

— Section authored by Lead CV Engineer

### 4. Methods Used

#### 4.1 Enhancement (Stage 1)

Each image is converted to grayscale, then gamma-corrected with a gamma chosen automatically from mean brightness (1.4 for dark images, 1.0 otherwise) so dark images are lifted without washing out images that are already bright. CLAHE (Contrast-Limited Adaptive Histogram Equalisation) is applied only when the image's contrast (pixel standard deviation) is below a threshold of 60 — on already-high-contrast images CLAHE was found to amplify texture noise (fur, fabric, foliage) to the same magnitude as genuine text edges, which confused the next stage. The image is finally denoised with Non-Local Means filtering, which preserves edges better than a Gaussian blur because it averages similar-texture patches rather than nearby pixels.

#### 4.2 Segmentation (Stage 2)

Three complementary binary masks are combined with a logical OR:

- **Adaptive (Gaussian) thresholding** — a locally-computed threshold that is robust to uneven lighting; block size auto-scales to image resolution and the foreground/background polarity is auto-detected and flipped if needed.
- **Sobel gradient magnitude** — captures the sharp edges produced by text strokes on busy or textured backgrounds.
- **Otsu global thresholding** — catches high-contrast text (e.g. white text on a dark sign) that local methods can under- or over-segment; its polarity is also auto-corrected by comparing mean foreground vs. background brightness.

### 4.3 Morphological Cleaning (Stage 3)

A sequence of classical morphological operators turns the noisy candidate mask into clean word-shaped blobs: **opening** (erosion + dilation) removes isolated speckles; an **erosion/dilation cycle** thins remaining noise while restoring stroke width — this cycle is skipped in favour of a single dilation when the mask is already dense ( $\geq 30\%$  white pixels), since a dense mask usually indicates large/bold text rather than noise; a wide-rectangular **closing** operator then bridges the gaps between adjacent characters so a whole word becomes one connected component; finally, **connected-component filtering** discards blobs that are too small, too large, too elongated, or that span almost the full image width/height (criteria that flag background edges or character silhouettes rather than text).

### 4.4 Detection (Stage 4)

OpenCV contour extraction converts the cleaned mask into bounding boxes. Each box then passes through three filters in sequence: a **geometry filter** (area, width/height minimums, aspect ratio, max image-width fraction), **Non-Maximum Suppression** by Intersection-over-Union to collapse duplicate/overlapping boxes, and an **isolated-noise filter** that discards small boxes with no other box nearby — on the reasoning that real text usually appears near other text (adjacent words, stacked lines), while a single small leftover noise blob does not.

### 4.5 Decision Logic (Stage 5)

Surviving boxes are classified as *word*, *line*, or *block* by width and aspect ratio, then the system issues a final verdict: text is declared "found" only if the box count meets a minimum **and** the boxes' combined area covers at least 1% of the image — a coverage floor that prevents a single stray 1-pixel survivor from triggering a false positive. The result is serialised to JSON alongside a human-readable verdict string.

— Sections 4.1–4.2 by Lead CV Engineer; 4.3 by Morphology & Visualization Lead; 4.4–4.5 by Lead CV Engineer

## 5. Results — Stage-by-Stage Example

The figure below traces one representative input (a comic panel with a speech bubble) through all six visualisation panels produced by the pipeline.

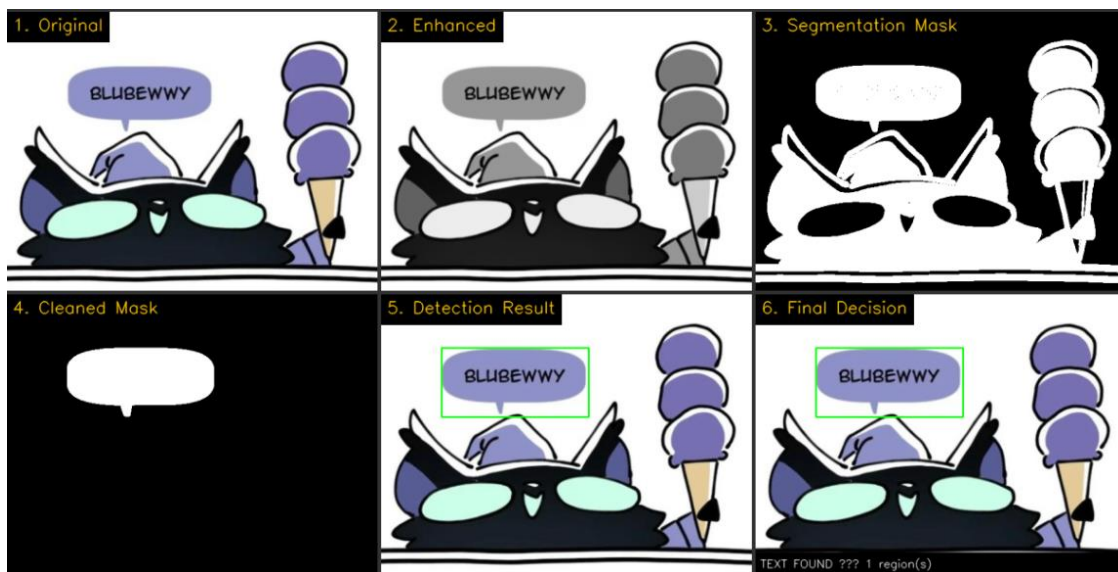


Figure 2. Stage-by-stage output: 1) Original 2) Enhanced 3) Segmentation mask 4) Cleaned mask 5) Detection result 6) Final decision overlay.

## Reading the example

Panel 3 (raw mask) shows exactly the difficulty described in Section 1: the segmentation stage by itself cannot distinguish the text strokes from the character's facial features or the ice-cream graphic — all three produce strong edges/contrast. The morphological cleaning stage (panel 4) is what actually solves the problem here, by exploiting the fact that text blobs are small, horizontally compact after closing, and bounded in aspect ratio, while the character's features and the ice-cream cone are not. The final verdict for this example was `TEXT FOUND - 1 region(s)`, correctly reporting exactly one text region (the speech bubble) and excluding every non-text shape in the frame.

Across the wider test set, the same pipeline was validated on photographs of street signs, social-media meme overlays, and scanned documents — image types with very different lighting, background busyness, and text density than the comic-panel example above, exercising the adaptive (rather than fixed) thresholds built into every stage.

— Section authored by Morphology & Visualization Lead

## 6. Failure Cases

Three recurring failure modes were identified during testing and diagnosed back to a specific stage. Each is described below, together with the fix that was implemented.

### 6.1 Background Blob Suppression

**Symptom:** on images with a large solid-coloured foreground subject (e.g. a character's body or a dark silhouette), the segmentation mask included the entire subject outline as one giant blob, which then survived early filtering and corrupted the density measurement used by later stages. **Fix:** connected-component filtering was moved to run *before* the erosion/dilation density decision rather than after, and explicit rejection rules were added for blobs covering more than ~70% of the image area or more than 60% of the image height — sizes no legitimate text region reaches.

### 6.2 Morphological Closing Artifacts

**Symptom:** the wide horizontal kernel used to merge individual characters into words could, on images with a large nearby non-text shape, also bridge the gap between real text and that shape — fusing them into a single oversized blob that detection then rejected outright (losing the text entirely). **Fix:** a two-pass adaptive closing strategy was introduced. When a dominant blob already covers more than

15% of the image (or already spans more than 60% of the image width), the pipeline first applies a small square kernel that only reconnects pixels *within* a character's own stroke, then a second, narrow horizontal kernel explicitly capped at ~3% of image width — wide enough to join adjacent letters, too narrow to reach across to the background shape.

### 6.3 Isolation-Filter False Negatives

**Symptom:** the noise-rejection rule that discards small, spatially isolated boxes was occasionally discarding *real* text — specifically short captions placed in opposite corners of an image (a common pattern in overlay-text images), where the two genuine text regions were far apart in absolute pixels and each looked "isolated" to the other. **Fix:** the neighbour-search radius was widened from 3× to 10× a box's own height, and the isolation filter is now skipped entirely whenever two or fewer boxes survive the earlier geometry filter — at that point, two boxes both surviving every prior filter are overwhelmingly likely to both be genuine text rather than coincidental noise.

All three failure modes were caught by dedicated unit tests written against synthetic masks before being fixed, and the same tests now run in regression to prevent the bugs from re-appearing as the filtering thresholds are tuned further.

— Section authored by Lead CV Engineer and Morphology & Visualization Lead (joint diagnosis and fixes)

## 7. Conclusion

The project shows that a carefully tuned classical CV pipeline — with no learned model anywhere in the loop — can reliably separate text regions from visually similar non-text content across a varied set of inputs, provided the segmentation stage is allowed to over-generate candidates and the morphological and geometric filtering stages are given enough adaptive logic to clean them up. The clean stage-to-stage contracts (fixed array shapes and value ranges) made independent development and debugging straightforward across the team, and the six-panel visualisation proved essential for diagnosing exactly which stage was responsible for a given failure.

The main limitation of the approach is that it has no notion of what text actually looks like — it only knows the geometric and statistical properties that *tend* to distinguish text from background. Inputs with unusually shaped logos, dense fine texture, or text that is itself curved or rotated remain to be the hardest cases. Natural next steps would be to feed the surviving regions from this pipeline into an OCR engine for text recognition, and to benchmark the classical detector against a lightweight learned text-detection model on the same test set to quantify where the rule-based approach starts to fall short.

— Section authored by the whole team